

Command line dumps

External utilities can be used to trigger heap dumps from the command line. For instance, *jmaptool* shipped with Oracle JDK can be used to dump memory of a locally running JVM process. In a development or test environment, this tool can be embedded into the code and dumps can be taken from within the program itself when a specific runtime condition occurs. Thread dumps on an Oracle JVM can be obtained using *jstacktool*, which is shipped with the Oracle JVM.

Programmatic dumps

A developer can put in JVM specific code at points in the program where the dump needs to be taken. This approach is useful only during development or during integration testing, when the developer still has control over the running program. Compared to the other methods of obtaining dumps described earlier, this is a less commonly used and lesser known technique as it involves modification to the source code and is very specific to the JVM provider and its version. This technique can be used when analysing large complex programs in a developer environment and the intent is to see kinds of objects created when the program execution hits a specific point in code.

On Oracle JVMs, the following code can be used to take a programmatic dump. In order to execute this code, *tools.jar* in *lib* directory of JDK installation - *\$JDK_HOME/lib/tools.jar* --should be present on the *classpath*.

```
//following technique for obtaining the process id works on
Windows machines. // A similar codecan be written for Unix
flavours to get the process id
```

```
String pid = ManagementFactory.getRuntimeMXBean().getName().
substring(0, name.indexOf('@'));
```

```
//set the options required for jmap class. set the values of
dumpDir and fileName as required
String jmapOption = "-dump:live,format=b,file=" + dumpDir
+ File.separatorChar + fileName;
```

```
String[] args = new String[2];
args[0] = jmapOption;
args[1] = pid;
//invoke jmap to take the dump
sun.tools.jmap.JMap.main(args);
```

On IBM JVMs, the following code can be used to take programmatic dumps. The *jar* file, *vm.jar*, in the JDK directory should be present on *classpath*. For instance, in the IBM Java SDK 1.7, this *jar* file would be present in *\$JDK_HOME/jre/bin/default/jcISC170*. On a Websphere Application Server (WAS) installation, this *jar* could be present in locations such as: *\$WAS_HOME/java/jre/bin/default/jcISC160*

```
com.ibm.jvm.Dump.HeapDump();//generates a heap dump in PHD
format
//
com.ibm.jvm.Dump.JavaDump(); //generates a java core dump a
.txt file
//OR
com.ibm.jvm.Dump.SystemDump(); //generates a system dump in
.dmp format
```

Terminology used in memory dumps

Garbage Collection Root (GC Root)

Garbage Collection Roots (GC Roots) are objects that are kept alive by the JVM. These objects are accessible from outside the heap and include:

- * The thread objects of the threads that are currently running.
- * The objects that are currently on the call stack.
- * The classes that were loaded by the system class loader.

GC Root queries are run to identify why a particular object is still live. It gives a chain of objects, tracing the object in question all the way to the root object that falls in one of the above criteria. The path helps in explaining why an object cannot be garbage collected and is useful in solving typical Java memory leak problems. Essentially, it helps in determining a memory leak by giving information

Query	Description
select x from com.perf.Person x	Lists all instances of the Person object on the heap
select x from com.perf.Person x where x.age<20	Lists all instances of the Person object on the heap whose age is less than 20
select filter(heap.classes(), /*com.perf./(it.name)*/))	Gives all the objects in the package and sub-packages of <i>com.perf</i>
select max(map(heap.objects('com.perf.Person', false), 'it.count'))	Finds out the maximum size of a Person object on the heap. The flag false indicates whether sub types of a Person, such as Employee and Customer should be included in the search.
sum(filter(heap.objects(), /*^com.perf.memory/(classof(it).name)*), 'sizeof(it)')	Finds the sum of all objects in the package - <i>com.perf.memory</i> . Function <i>sizeof</i> stands for retained size and may not be supported in all OQL tools.